NEXUS Programming Language — Manual

Version 2.0.0

1. Introduction

NEXUS is a native programming language with a C++ compiler, an LLVM IR backend, a small native backend, a runtime library, and a standard library written in NEXUS. Programs can be compiled and run with:

```
nexus run file.nx --backend llvm
```

Type checking is available with:

```
nexus check file.nx
```

2. Quick start

Create hello.nx:

```
fn main() {
    print("Hello, NEXUS!")
    print(40 + 2)
}
```

Run it:

```
nexus run hello.nx --backend llvm
```

3. Lexical structure

- Line comments start with //.
- Block comments use /* ... */.
- Identifiers use letters, digits, and underscores.
- Keywords include fn, let, mut, return, if, else, while, print, true, and false.

4. Types

NEXUS supports the following basic types:

- i64 – signed 64-bit integer
- f64 – 64-bit floating point number
- bool – boolean value
- string – text value

Integer literal:

```
let a = 42
```

Float literal:

```
let x = 3.5
```

String literal:

```
let s = "hello"
```

Boolean literal:

```
let ok = true
```

5. Variables

Declare an immutable variable:

```
let x = 10
```

Declare a mutable variable:

```
let mut i = 0
```

Assign to a mutable variable:

```
i = i + 1
```

Variables must be declared before use. Assignment does not change a variable type.

6. Operators

Arithmetic:

```
a + b
a - b
a * b
a / b
a % b
```

Comparisons:

```
a == b
a ≠ b
a < b
a ≤ b
a > b
a ≥ b
```

Logical operators:

```
a && b
a || b
```

Unary operators:

```
-x
!flag
```

String concatenation:

```
let s = "hello" + " " + "world"
```

7. Control flow

If statement:

```
if x > 0 {
    print("positive")
} else {
    print("not positive")
}
```

While loop:

```
let mut i = 0
while i < 5 {
    print(i)
    i = i + 1
}
```

8. Functions

Define a function:

```
fn add(a: i64, b: i64) → i64 {
    return a + b
}
```

Call a function:

```
print(add(40, 2))
```

Functions may return i64, f64, bool, or string. A function without an explicit return type may be used for procedures. Functions should end with return or receive an automatic return value where supported.

Main function:

```
fn main() {
    print("Hello")
}
```

9. Strings

Print a string:

```
print("hello")
```

Concatenate strings:

```
print("a" + " " + "b")
```

String values can be passed to runtime functions through the standard library and FFI where available.

10. Floating point

Floating point values use f64:

```
let x = 1.5
let y = 2.0
print(x + y)
print(x * y)
```

Comparisons and arithmetic work on f64 values.

11. Standard library

The standard library is located in stdlib/. Common modules include:

- io — input and output helpers
- fs — file system helpers
- string — string helpers
- os — operating system helpers
- json — JSON helpers
- db — database helpers
- net — networking helpers
- crypto — cryptographic helpers
- memory — memory helpers

Import modules with:

```
import "io.nx"
```

12. Building and running

Build the compiler:

```
cmake --build build -j
```

Run a program:

```
./build/nexus.exe run file.nx --backend llvm
```

Check types:

```
./build/nexus.exe check file.nx
```

Run tests:

```
ctest --test-dir build
```

13. Native backend

The native backend supports only a small subset of the language. For serious programs, use --backend llvm.

14. Self-hosting

The selfhost/ directory contains a lexer, parser, type checker, and code generator written in NEXUS. The self-hosted code generator currently supports the documented subset: let, assignment, print, arithmetic, comparisons, logical operators, if, while, functions, calls, strings, and f64.

15. Examples

A simple function example:

```
fn mul(a: i64, b: i64) → i64 {
    return a * b
}

fn main() {
    print(mul(6, 7))
}
```

A floating point example:

```
fn main() {
    let x = 1.5
```

```
    let y = 2.0
    print(x + y)
}
```

16. Notes

- Functions must be declared before use.
- Use let mut for mutable variables.
- let _ = ... discards a value.
- Native backend is limited; use LLVM backend for full programs.